

Mitigating Cyberbullying with TriFuse: A Multi-View Model Fusing Lexical, Semantic, and Structural Text Features

Krish Monga¹, Shivansh Mahajan², & Pardeep Garg^{3*}

^{1,2}Dept. of Computer Science Engineering & Information Technology
Jaypee University of Information Technology

Waknaghat, Solan, H.P. 173234, India

¹E-mail: krishmonga21667@gmail.com

²E-mail: shivanshmahajan255@gmail.com

^{3*}Dept. of Electronics and Communication Engineering

Jaypee University of Information Technology

Waknaghat, Solan, H.P. 173234, India

E-mail: pardeep.garg@juitsolan.in

Corresponding author

Abstract—Manual review cannot keep pace with abusive content on large social media feeds, especially when harassment is indirect or platform-specific. Many existing classifiers depend on one feature type alone (n-grams, sequence order, or contextual embeddings) and overlook signals captured by the others. We propose TriFuse, a multi-view network that learns lexical, semantic, and structural features together. Phrase cues are extracted with a CNN branch, wider context with a Transformer branch, and token order with a BiLSTM branch; cross-view interaction and temperature-scaled attention merge the three representations inside the model instead of combining separate classifiers at the end. Evaluation uses the Shahane multi-platform corpus (Dataset I, 229,228 instances) and the Davidson hate-speech corpus (Dataset II, 24,783 instances) with shared preprocessing, splits, and training budgets. On Dataset I, TriFuse reaches 94.32% test accuracy, 94.12% F1, and $94.29 \pm 0.13\%$ under stratified 5-fold cross-validation, ahead of Random Forest, LightGBM, CNN, BiLSTM, and Tuned LSTM. On Dataset II, it reaches 95.87% accuracy, 95.91% F1, and $95.37 \pm 0.23\%$ cross-validated accuracy with the same ordering. Ablation runs on both corpora show that the full model beats single-branch, pairwise, and late-fusion variants. Improvements over the strongest neural baselines are small but repeat across folds; paired tests, scope limits, and training cost are discussed in Section VII.

Index Terms—Cyberbullying detection, multi-view learning, attention fusion, deep learning, natural language processing, cross-dataset evaluation, online safety

I. INTRODUCTION

Cyberbullying has become more common as social media use has grown, and automated screening is needed to limit the spread of harmful posts [1]. Early systems relied on hand-crafted features with classical classifiers [2]. Deep learning methods were later applied to abusive-language detection [3], but performance still drops when wording changes across platforms or when abuse is implied rather than explicit. Sev-

eral public benchmarks for harmful-content detection already report high headline accuracy [4].

Random Forest [5] and LightGBM [6] are still widely used baselines. They work on fixed feature vectors and do not model word order or long-range dependencies well. CNNs pick up local n-gram patterns in short text [7], and LSTMs model token sequences [8], yet each architecture emphasizes one representation type. Transformers improve semantic context [9], but a single encoder stream may still underuse lexical and structural cues.

TriFuse combines these views in one network. Three encoders (CNN, Transformer, and BiLSTM) process shared token embeddings, interact through cross-view attention, and are merged with learned branch weights. Training uses Focal Loss for class imbalance, AdamW optimization, and auxiliary losses that encourage branch diversity. We report results on Dataset I (Shahane) [10] and Dataset II (Davidson) [11] against five baseline models under the same preprocessing pipeline and split policy [12], [13].

Our contributions are:

- 1) A single end-to-end model that joins lexical, semantic, and structural encoders, unlike stacking ensembles [14] or hybrid models that depend on separately engineered lexical stacks [15], [16].
- 2) Representation-level fusion with cross-view attention and temperature scaling to weight each branch during classification.
- 3) Evaluation on two public corpora (multi-platform Shahane and Twitter hate-speech Davidson) with stratified 5-fold cross-validation on both.
- 4) Ablation experiments for single branches, branch pairs, and late fusion to measure each design choice.
- 5) Released split settings (seed 42) and training configura-

tions to support replication on the same public data.

Section II reviews related work. Sections III and IV describe the datasets, preprocessing, and TriFuse design. Section V lists baseline models, Section VI the training protocol, Section VII the main results, Section VIII ablations, and Section IX the conclusion.

II. LITERATURE REVIEW

A. Cyberbullying Detection in Social Media

Cyberbullying research reports serious effects on mental health, especially among adolescents [1]. Survey papers note that detection remains difficult because of sarcasm, missing context, and inconsistent labels across sources [12]. Deep models have therefore been adopted to reduce reliance on hand-tuned features [13].

Recent work stacks Transformers with secondary classifiers [14], augments contextual models with lexical features [15], [16], or models post relations with graph networks [17]. These systems report strong scores, but fusion is often applied after separate training rather than inside one shared encoder stack.

B. Traditional Machine Learning Methods

Older detectors used Random Forest (RF) [5], LightGBM [6], or Naive Bayes (NB) [18] on bag-of-words or TF-IDF vectors. They train quickly and are easy to inspect, but they miss long context and often fail when the vocabulary shifts between platforms.

C. Deep Learning for Text Representation

Deep networks changed how short social text is modeled [3], [7], [8], [13]. CNNs learn local n-grams; LSTMs track order; hybrid CNN-LSTM designs mix both views; attention highlights tokens that drive the label. In practice, many cyberbullying classifiers still train one representation family per model.

D. Transformer-Based Contextual Modeling

Transformer architectures model long-range dependencies through self-attention, allowing each token to draw on full-sequence context rather than fixed local windows [9]. Pre-trained encoders now provide strong semantic representations across many NLP tasks. Used alone, however, a single Transformer stream may still underuse phrase-level and order-based cues common in abusive posts.

E. Multi-View and Representation Learning

Multi-view learning combines different feature spaces to improve generalization [19]. Multi-task setups can share layers across objectives [20]. Other social-text studies have combined embeddings with linguistic or profile-level signals [21], [22], but cyberbullying detectors often rely on late concatenation or voting instead of learned mid-level fusion.

F. Training Objectives and Optimizers

Abusive classes are usually outnumbered in public corpora, which biases accuracy toward the negative label. Focal Loss down-weights easy examples so harder abusive cases receive more gradient [23]. Adam [24], AdamW [25], and cosine restart schedules are common optimizers for text classifiers.

TABLE I
COMPARATIVE SUMMARY OF STATE-OF-THE-ART CYBERBULLYING
DETECTION APPROACHES

Ref	Model	Representation	Perf. (Acc./F1)	Limitation
[2]	Feature ML	Lexical	73.9% (F1)	Limited context
[14]	Transformer Ens.	Contextual	97.4% (Acc.)	Ensemble-based
[15]	RoBERTaNET	Contextual+Lexical	95.0% (Acc.)	External feature stack
[17]	SOSNet	Graph-based	92.7% (Acc.)	Fine-grained multiclass
[26]	CNN+Attn	Lexical	94.0% (Acc.)	No structural modeling
[31]	Feature ML	Lexical	85.5% (Acc.)	Single-platform tweets

Note: Acc. = accuracy; F1 = F1-score. Ref. 2 reports F1 on hate-speech features; Refs. 14, 15, 26, and 31 report accuracy; Ref. 17 reports SBERT+SOSNet accuracy on a fine-grained five-class subset of 4,000 tweets (Ref. 26: 93.96% in source, rounded). Datasets, splits, tasks, and preprocessing differ, so values are not directly comparable with Tables IV and VIII.

Table I summarizes representative prior systems and their reported limits; graph, CNN-attention, ensemble-transformer, embedding-based, and textual-feature pipelines are discussed in [26], [27], [28], [29], [30], [31].

G. Research Gaps

The review points to five open issues:

- 1) Classical models capture limited context [5], [6], [18].
- 2) CNN and LSTM baselines usually optimize one feature type [7], [8].
- 3) Single-stream Transformers underuse lexical and structural cues [9].
- 4) Hybrid papers often fuse at the decision layer [14] or rely on externally engineered feature stacks [15], [16].
- 5) Joint attention fusion over lexical, semantic, and structural views remains rare in cyberbullying work.

TriFuse addresses these points by encoding all three views in one network, fusing them before the softmax layer, and training with Focal Loss on two public benchmarks.

III. DATASET AND PREPROCESSING

A. Datasets

We use two public corpora under one binary label: abusive versus non-abusive text. The term *cyberbullying detection* refers to that task in this paper. Source labels include aggression, toxicity, hate speech, and offensive language, but all positive cases are mapped to a single abusive class so models can be trained and compared on the same objective [12], [13].

1) Dataset I: Aggregated Shahane Cyberbullying Corpus

Dataset I is built from the Shahane cyberbullying release on Kaggle [10]. It merges seven platform-specific CSV subsets (aggression, attack, toxicity, Twitter racism/sexism, YouTube, and related Kaggle splits) into one file list, following multi-source bullying corpora described in prior work [12], [27].

Each subset provides textual data and corresponding probabilistic (`ed_label_0`, `ed_label_1`) and/or binary (`oh_label`) annotations. Probabilistic labels provide confidence scores for the non-bullying and bullying classes, respectively, while `oh_label` provides the final binary ground truth label when available. To align mixed annotation schemes, all samples are mapped to one binary label:

$$y = \begin{cases} 1 & \text{Abusive} \\ 0 & \text{Non-abusive} \end{cases} \quad (1)$$

TABLE II
DATASET EXAMPLES (DATASET I)

Text	ed_label_0	ed_label_1	oh_label
"This is not creative. Those are the dictionary definitions of the terms insurance..."	0.9	0.1	0
"You are so stupid and worthless!"	0.0	1.0	1
"I hate you, go die!"	0.0	1.0	1

TABLE III
DATASET CHARACTERISTICS

Property	Dataset I (Shahane)	Dataset II (Davidson)
Source	Kaggle multi-subset corpus [10]	Twitter hate-speech corpus [11]
Role	Primary evaluation corpus	Cross-corpus generalization
Instances (prepared)	229,228	24,783
Abusive instances	27,520	20,620
Non-abusive instances	201,708	4,163
Label space (unified)	Abusive / non-abusive	Abusive / non-abusive
Class imbalance	Present (1:7.3)	Present (1:4.9)

When only probability scores exist, the higher-scoring label is kept. After preprocessing, empty rows are removed and duplicate posts are dropped. The final Dataset I split contains 229,228 posts (27,520 abusive, 201,708 non-abusive), covering insults, hate speech, aggression, and toxic comments. Class imbalance follows prior reports on this corpus [12]. Table II shows sample rows.

2) Dataset II: Davidson Hate-Speech Corpus

Dataset II comes from Davidson et al. [11], a Twitter corpus labelled as hate speech, offensive language, or neither. We map hate speech and offensive language to abusive ($y = 1$) and neither to non-abusive ($y = 0$) so the label definition matches Dataset I. The prepared file contains 24,783 tweets (20,620 abusive, 4,163 non-abusive). Because the source platform and annotation scheme differ from Shahane, Dataset II serves as an out-of-domain check rather than a relabelled copy of Dataset I.

Both corpora contain informal user text from social platforms, so they expose lexical, semantic, and structural models to realistic noise [8], [9], [19].

B. Preprocessing Pipeline

Fig. 1 outlines the shared preprocessing steps used for social text cleanup [12]. The same code path is applied to Dataset I and Dataset II.

1) Text Cleaning and Normalization

Text is lowercased, URLs are mapped to [URL], and @mentions to [USER]. Common emoticons are replaced with words (e.g., ":-)" → "smile", "<3" → "love"). Character runs longer than two are shortened ("looooooser" → "looser"). Punctuation that may carry sentiment (period, comma, !, ?) is kept; other symbols are stripped. Extra spaces are collapsed to one blank.

2) Tokenization

Cleaned text was tokenized using NLTK-based word tokenization. Given an input sequence:

$$X = \{w_1, w_2, \dots, w_n\}$$

where w_i denotes the i^{th} token.

3) Vocabulary Development and Coding

The vocabulary was created from the training corpus using frequency-based selection. Out-of-vocabulary tokens were mapped to an unknown token (UNK). Special tokens <PAD>, <UNK>, <BOS>, and <EOS> were added. Each tokenized

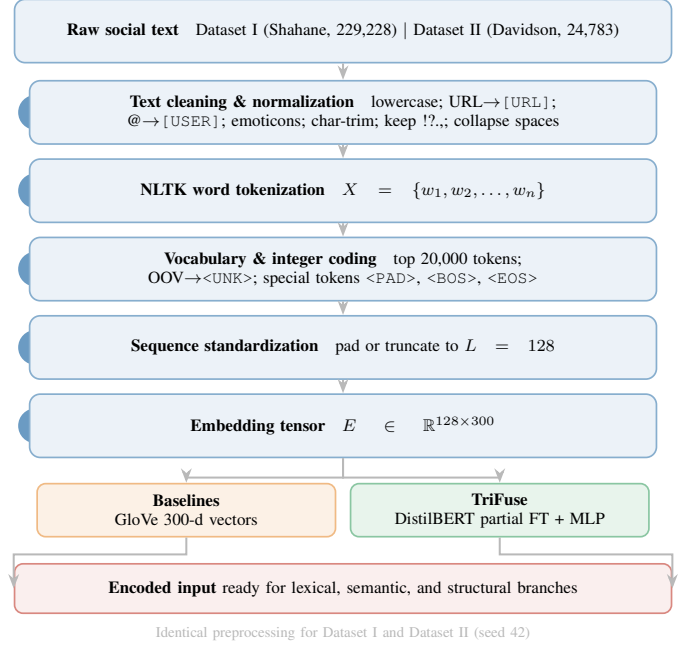


Fig. 1. Text preprocessing pipeline

sequence was transformed into a fixed-length numerical representation for batch processing.

4) Standardization of Sequence Length

For batched training across TriFuse and the neural baselines, each tokenized sequence was padded with <PAD> or truncated to a uniform maximum length of $L = 128$ tokens.

5) Word Embedding Representation

For neural baselines, word vectors were initialized with pre-trained GloVe embeddings as widely used in text classification and abusive language detection [28], [29]. Each token was mapped to a 300-dimensional dense vector:

$$E \in \mathbb{R}^{128 \times 300}.$$

TriFuse additionally receives contextual token embeddings from a partially fine-tuned DistilBERT encoder (distilbert-base-uncased); these embeddings are projected to 300 dimensions and shared across all three view encoders.

6) Handling Class Imbalance

Both corpora contain far more non-abusive than abusive posts [12]. We keep the original counts and apply Focal Loss [23] so misclassified abusive examples receive higher loss during training.

IV. PROPOSED METHODOLOGY

Fig. 2 shows TriFuse. Lexical, semantic, and sequential features are computed in parallel and merged before classification. Unlike single-stream baselines [7], [8] or late ensembles [14], the three views share gradients through mid-level fusion layers.

We define the input sequence as:

$$X = \{w_1, w_2, \dots, w_L\}$$

where $L = 128$ represents a fixed sequence length after preprocessing. The objective is to learn a function:

$$f : X \rightarrow y, \quad y \in \{0, 1\} \quad (2)$$

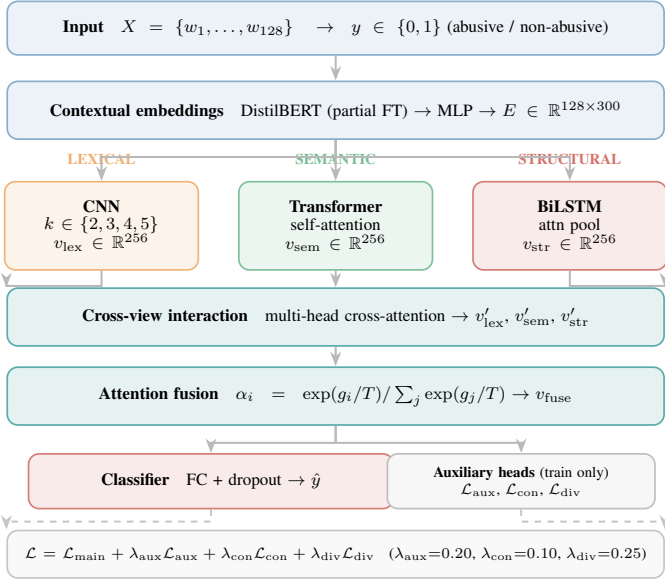


Fig. 2. TriFuse architecture

representing the cyberbullying label.

A. Feature Engineering

Offensive posts can surface through word choice, surrounding context, or token order [7], [8], [9]. TriFuse therefore builds three vectors before fusion:

- 1) Lexical features
- 2) Semantic context
- 3) Sequential structure

Formally, the feature extraction process learns three representations:

$$v_{\text{lex}}, v_{\text{sem}}, v_{\text{str}}$$

which are subsequently fused for final classification.

1) Lexical Feature Extraction

Lexical features capture local n -gram patterns and phrase-level abusive indicators [7]. Let the embedding matrix be $E \in \mathbb{R}^{L \times D}$ with $L = 128$ and $D = 300$. A 1D convolution with filter size k is defined as:

$$\begin{aligned} c_i &= f(W_k \cdot E_{(i:i+k-1)} + b_k) \\ \hat{c} &= \max(c) \end{aligned} \quad (3)$$

where W_k denotes the convolution kernel, b_k is bias, and $f(\cdot)$ is ReLU. Multiple filter sizes $k \in \{2, 3, 4, 5\}$ with 64 filters each capture bi-grams, tri-grams, and higher-order lexical patterns. The final lexical representation is $v_{\text{lex}} \in \mathbb{R}^{256}$.

2) Semantic Contextual Feature Extraction

The Transformer encoder models token interactions with self-attention [9]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (4)$$

where Q , K , and V denote query, key, and value matrices, and d_k is the scaling dimension. The semantic encoder outputs $v_{\text{sem}} \in \mathbb{R}^{256}$ by pooling information across the full sequence.

3) Structural Sequential Feature Extraction

Word order can change the meaning of a hostile post. For each time step t :

$$\begin{aligned} h_t &= \text{LSTM}(e_t, h_{t-1}) \\ h_t &= [\vec{h}_t; \overleftarrow{h}_t] \\ \alpha_t &= \frac{\exp(u_t)}{\sum_{j=1}^L \exp(u_j)}, \quad v_{\text{str}} = \sum_{t=1}^L \alpha_t h_t \end{aligned} \quad (5)$$

where bidirectional modeling captures forward and backward dependencies, and an attention mechanism [30] highlights informative time steps. The resulting structural representation is $v_{\text{str}} \in \mathbb{R}^{256}$.

B. Contextual Embedding Layer

TriFuse employs distilbert-base-uncased as a shared contextual embedding front end. Input text is tokenized with the DistilBERT tokenizer; all encoder layers except the final transformer block are frozen, and only that last block is fine-tuned at one hundredth of the base learning rate. Hidden states are projected to $D = 300$ through a two-layer MLP with layer normalization and GELU activation; this projection layer is trained at the base learning rate and the resulting embeddings are supplied to the lexical, semantic, and structural branches. This DistilBERT module provides token-level context and is separate from the trainable Transformer stack in the semantic branch below. Vocabulary indices and GloVe initialization remain used for baselines and ablation variants that disable the DistilBERT front end.

C. Lexical Feature Encoding (CNN Branch)

As detailed in Fig. 3, a 1D CNN extracts n -gram features and abusive word information locally.

The feature map for a k -sized convolution kernel is:

$$\begin{aligned} c_i^{(k)} &= \text{ReLU}(W_k \cdot E_{i:i+k-1} + b_k) \\ \hat{c}^{(k)} &= \max_i c_i^{(k)} \end{aligned} \quad (6)$$

where $k \in \{2, 3, 4, 5\}$. The concatenated features create the lexical representation $v_{\text{lex}} \in \mathbb{R}^{256}$.

D. Semantic Context Encoding (Transformer Branch)

Beyond the DistilBERT front end, a dedicated Transformer encoder stack processes the shared embeddings and models long-range dependencies through self-attention [9]. The feed-forward component applies GELU activation:

$$\text{GELU}(x) = x \Phi(x) \quad (7)$$

where $\Phi(x)$ represents the CDF of the standard normal distribution. Mean and max pooling produce $v_{\text{sem}} \in \mathbb{R}^{256}$.

E. Structural Sequential Encoding (BiLSTM Branch)

A bidirectional LSTM [8] models sequential dependencies. At time step t :

$$\begin{aligned} \vec{h}_t &= \text{LSTM}(e_t, \vec{h}_{t-1}) \\ \overleftarrow{h}_t &= \text{LSTM}(e_t, \overleftarrow{h}_{t+1}) \\ h_t &= [\vec{h}_t; \overleftarrow{h}_t] \end{aligned} \quad (8)$$

Attention pooling yields $v_{\text{str}} \in \mathbb{R}^{256}$.

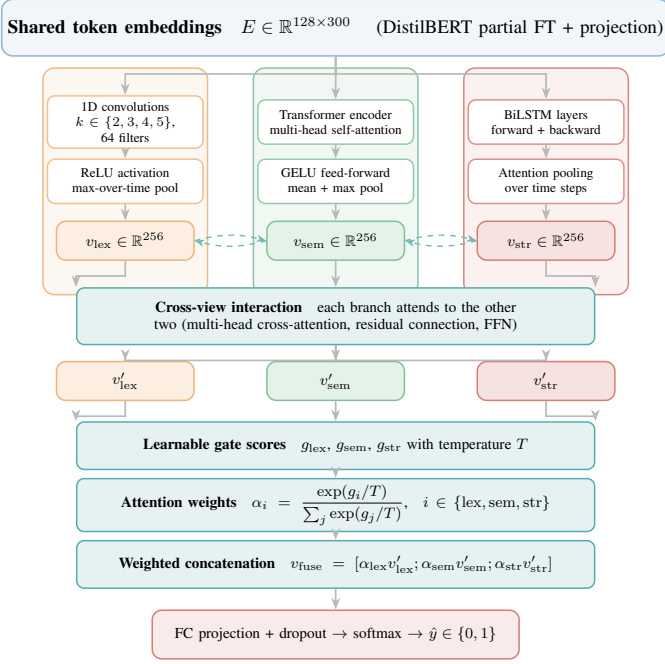


Fig. 3. TriFuse encoding and fusion flow: shared embeddings enter lexical, semantic, and structural branches; cross-view modules update each branch before fusion forms v_{fuse} .

F. Cross-View Interaction and Multi-Branch Feature Fusion

Each view is enriched by attending to the other two through multi-head cross-attention with residual connection and feed-forward sublayer. The three enriched representations are fused through attention, as visualized in the lower portion of Fig. 3:

$$\alpha_i = \frac{\exp(g_i/T)}{\sum_{j=1}^3 \exp(g_j/T)}, \quad i \in \{\text{lex}, \text{sem}, \text{str}\} \quad (9)$$

where g_i is the gate score for view i and T is a learnable temperature. The fused vector is

$$v_{\text{fuse}} = [\alpha_{\text{lex}} v'_{\text{lex}}; \alpha_{\text{sem}} v'_{\text{sem}}; \alpha_{\text{str}} v'_{\text{str}}]. \quad (10)$$

G. Fully Connected Projection and Classification

The fused feature is processed by a fully connected projection layer with dropout, and the prediction is computed as:

$$\begin{aligned} z &= \text{ReLU}(W_f v_{\text{fuse}} + b_f) \\ \hat{y} &= \text{softmax}(W_c z + b_c) \end{aligned} \quad (11)$$

where W_f , b_f , W_c , and b_c are learnable parameters.

During training only, auxiliary heads on each enriched view provide branch-level supervision. The total TriFuse loss is

$$\mathcal{L} = \mathcal{L}_{\text{main}} + \lambda_{\text{aux}} \mathcal{L}_{\text{aux}} + \lambda_{\text{con}} \mathcal{L}_{\text{con}} + \lambda_{\text{div}} \mathcal{L}_{\text{div}}, \quad (12)$$

with $\lambda_{\text{aux}} = 0.20$, $\lambda_{\text{con}} = 0.10$, and $\lambda_{\text{div}} = 0.25$.

H. Architectural Summary

TriFuse:

- Uses a partially fine-tuned DistilBERT (distilbert-base-uncased) front end to supply shared token embeddings to all three branches.
- Extracts lexical features through a CNN with ReLU activation.
- Models contextual relationships through a Transformer encoder with GELU activation.
- Models sequential relationships through a bidirectional LSTM with attention pooling.

- Enriches each view through cross-view interaction and fuses representations with learnable attention weighting.
- Produces predictions through a fully connected layer and softmax activation.

V. BASELINE MODELS

We compare TriFuse against five baselines from classical and neural text classification [3], [5], [6], [7], [8], [31]. Each baseline isolates one standard modeling choice.

A. Bidirectional Long Short-Term Memory (BiLSTM)

Bidirectional LSTM networks model dependencies with respect to both future and past context. The LSTM cell has gating functions:

$$\begin{aligned} f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \\ i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) \\ \tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) \\ h_t &= o_t \odot \tanh(c_t) \end{aligned} \quad (13)$$

In the bidirectional configuration, forward and backward hidden states are concatenated for classification.

B. Convolutional Neural Network (CNN)

The CNN classifier uses the sentence classification architecture defined by Kim [7]. Given a filter $W \in \mathbb{R}^{k \times D}$, convolution is computed as:

$$c_i = f(W \cdot x_{i:i+k-1} + b) \quad (14)$$

where k is the kernel width, $f(\cdot)$ is ReLU, and max-over-time pooling selects the most salient feature from each feature map.

C. Tuned LSTM

The Tuned LSTM baseline is a single-directional LSTM with higher hidden dimensionality, dropout regularization, and three layers:

$$h_t = \text{LSTM}(x_t, h_{t-1}).$$

This baseline checks whether a deeper unidirectional LSTM alone is sufficient for the binary task.

D. Random Forest (RF)

Random Forest [5] is an ensemble learning method that aggregates decision trees trained on bootstrap samples with random feature selection. The final prediction is obtained through majority voting:

$$\hat{y} = \text{mode}\{T_m(X)\}_{m=1}^M \quad (15)$$

Random Forest acts as a non-neural baseline using TF-IDF features.

E. LightGBM

LightGBM [6] is a gradient boosting framework that minimizes a differentiable loss function sequentially:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x) \quad (16)$$

where $F_m(x)$ is the model at iteration m , $h_m(x)$ is a weak learner, and η is the learning rate. It provides a strong ensemble baseline on TF-IDF features.

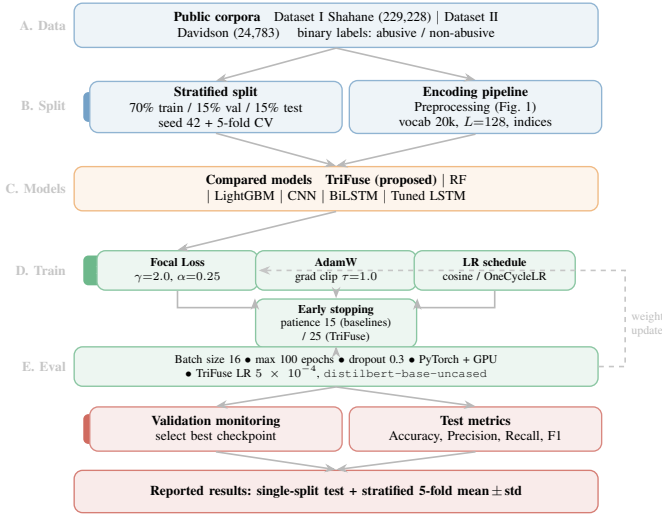


Fig. 4. Experimental training and evaluation pipeline.

VI. EXPERIMENTAL SETUP

All models share the workflow in Fig. 4: data preparation, token encoding, training with early stopping, and reporting accuracy, precision, recall, and F1 on the same held-out splits.

A. Data Splitting and Encoding

Each dataset was split using stratified sampling, preserving class priors across train, validation, and test subsets:

$$P_{\text{train}}(y) \approx P_{\text{val}}(y) \approx P_{\text{test}}(y).$$

The default partition is 70% training, 15% validation, and 15% test with random seed 42. Stratified 5-fold cross-validation was also run on the full dataset for every model to reduce split variance. All text inputs were preprocessed, tokenized, and indexed. Sequences were padded or truncated to maximum length $L = 128$. The vocabulary was built from the 20,000 most common tokens in the training corpus.

B. Optimization and Learning Schedule

Training used the AdamW optimizer [25]:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} - \lambda \theta_t \quad (17)$$

where η denotes the learning rate, $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected moment estimates, and λ is the weight decay coefficient. Baseline neural models used initial learning rate 10^{-3} with cosine annealing warm restarts:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{T_{\text{cur}}}{T_i} \pi \right) \right) \quad (18)$$

TriFuse used learning rate 5×10^{-4} , patience 25, OneCycleLR scheduling, and higher learning rates for cross-view and attention parameters. The DistilBERT front end followed the partial fine-tuning setup in Section IV-B. Gradient norms were clipped with threshold $\tau = 1.0$:

$$g \leftarrow \frac{g}{\max \left(1, \frac{\|g\|_2}{\tau} \right)}.$$

C. Loss Function

Focal Loss [23] emphasized minority instances that are hard to classify:

$$\text{FL}(p_t) = -\alpha(1 - p_t)^\gamma \log(p_t) \quad (19)$$

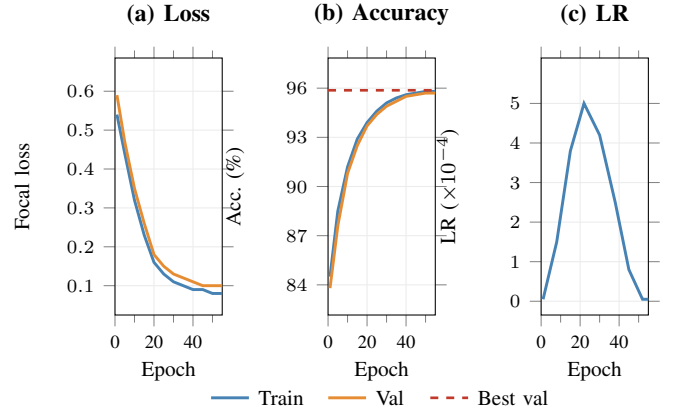


Fig. 5. TriFuse training dynamics on Dataset II (AdamW, batch size 16, patience 25, OneCycleLR).

TABLE IV
BASELINE MODEL COMPARISON: SINGLE-SPLIT TEST RESULTS

Model	Dataset I (Shahane)				Dataset II (Davidson)			
	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1
Random Forest	92.28%	91.56%	92.28%	91.10%	92.91%	92.68%	92.91%	92.51%
LightGBM	87.88%	92.02%	87.88%	89.23%	94.73%	95.66%	94.73%	94.96%
CNN	93.93%	93.50%	93.93%	93.52%	95.82%	95.92%	95.82%	95.86%
BiLSTM	94.28%	94.00%	94.28%	94.08%	95.71%	96.03%	95.71%	95.81%
Tuned LSTM	94.18%	93.80%	94.18%	93.77%	95.84%	95.98%	95.84%	95.89%
TriFuse (Proposed)	94.32%	94.03%	94.32%	94.12%	95.87%	95.96%	95.87%	95.91%

where p_t denotes the predicted probability of the actual class, $\gamma = 2.0$, and $\alpha = 0.25$.

D. Training Configuration

All neural models were trained for a maximum of 100 epochs with early stopping if validation accuracy did not improve for 15 consecutive epochs (25 for TriFuse). Batch size was 16. Dropout rate was 0.3. All experiments used PyTorch with GPU acceleration when available and random seed 42 for reproducibility. The best-performing checkpoint on the validation set was used for test evaluation. Fig. 5 illustrates representative loss, accuracy, and learning-rate dynamics for TriFuse on Dataset II under this configuration.

E. Evaluation Metrics

Performance was evaluated using accuracy, precision, recall, and F1-score:

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\ \text{Precision} &= \frac{TP}{TP + FP} \\ \text{Recall} &= \frac{TP}{TP + FN} \\ \text{F1} &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \end{aligned} \quad (20)$$

where TP , TN , FP , and FN denote true positives, true negatives, false positives, and false negatives, respectively.

VII. RESULTS AND DISCUSSION

Tables IV and V report single-split and 5-fold scores for TriFuse and the five baselines on both corpora under the same splits and training budget.

A. Baseline Model Comparison

Table IV and Fig. 6 report single-split test results for all models on both datasets.

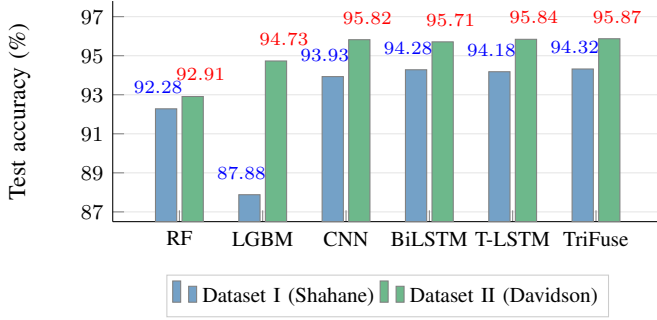


Fig. 6. Baseline and TriFuse test accuracy on Dataset I and Dataset II.

TABLE V
STRATIFIED 5-FOLD CROSS-VALIDATION RESULTS (MEAN $\pm$ STD)

Model	Dataset I (Shahane)		Dataset II (Davidson)	
	Accuracy	F1-score	Accuracy	F1-score
Random Forest	92.33 $\pm$ 0.05%	91.21 $\pm$ 0.04%	93.06 $\pm$ 0.32%	92.71 $\pm$ 0.35%
LightGBM	87.70 $\pm$ 0.19%	89.09 $\pm$ 0.17%	94.58 $\pm$ 0.40%	94.82 $\pm$ 0.37%
CNN	93.96 $\pm$ 0.08%	93.60 $\pm$ 0.15%	95.32 $\pm$ 0.35%	95.36 $\pm$ 0.38%
BiLSTM	94.24 $\pm$ 0.10%	94.05 $\pm$ 0.14%	95.28 $\pm$ 0.49%	95.26 $\pm$ 0.55%
Tuned LSTM	94.22 $\pm$ 0.08%	94.14 $\pm$ 0.06%	95.31 $\pm$ 0.52%	95.32 $\pm$ 0.59%
TriFuse (Proposed)	94.29 $\pm$ 0.13%	94.07 $\pm$ 0.27%	95.37 $\pm$ 0.23%	95.41 $\pm$ 0.24%

TABLE VI
SINGLE-BRANCH PERFORMANCE

Variant	Dataset I (Shahane)				Dataset II (Davidson)			
	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1
Lexical Only	93.99%	93.80%	93.99%	93.88%	95.90%	95.84%	95.84%	95.87%
Semantic Only	92.99%	93.23%	92.99%	93.10%	95.46%	95.66%	95.46%	95.53%
Structural Only	93.91%	93.53%	93.91%	93.59%	95.50%	95.57%	95.57%	95.53%
TriFuse (Full)	94.32%	94.03%	94.32%	94.12%	95.87%	95.96%	95.87%	95.91%

Note: Single-branch variants, including structural-only (isolated BiLSTM branch), use the same splits, seed, and training budget as the ablation study in Section VIII.

TriFuse ranks first in every table we report, but the margins over CNN, BiLSTM, and Tuned LSTM are small (0.03–0.16 points in test accuracy). On Dataset II, its 5-fold accuracy spread is 0.23%, lower than CNN (0.35%) and Tuned LSTM (0.52%), yet paired significance tests on fold scores did not reject the null at $\alpha = 0.05$ (Section VII-D). The results are therefore best read as a stable ranking under a fixed protocol rather than a large effect size.

B. Discussion of Baseline Results

On Shahane, TriFuse scores 94.32% test accuracy, above BiLSTM (94.28%) and well above the tree-based models. On Davidson, it reaches 95.87%, ahead of CNN (95.82%), Tuned LSTM (95.84%), and BiLSTM (95.71%). Cross-validation gives 94.29 $\pm$ 0.13% on Dataset I and 95.37 $\pm$ 0.23% on Dataset II; among deep nets, TriFuse has the smallest accuracy spread on Dataset II.

Table VI shows that no single branch matches the full model. CNN, BiLSTM, and Tuned LSTM each cover one view, whereas TriFuse trains all three with shared fusion layers and finishes higher on both datasets.

C. Multi-View Representation Analysis

Table VI and Fig. 7 report single-branch configurations on both datasets.

The lexical branch already reaches 93.99% on Dataset I and 95.84% on Dataset II, so local n-gram cues carry much of the signal. The semantic branch scores lowest (92.99% and 95.46%), while the structural branch sits between them (93.91% and 95.57%). Full TriFuse still tops every column, which suggests the branches capture partly non-overlapping information even when single-view scores are close.

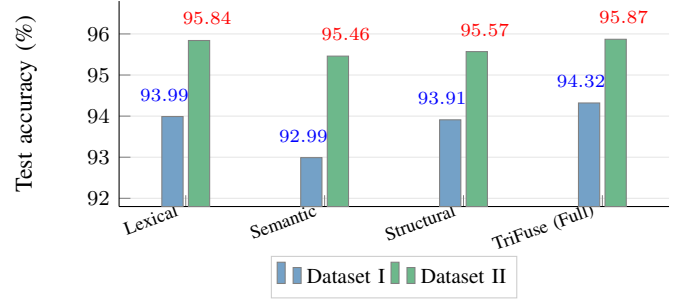


Fig. 7. Test accuracy for single-branch variants and full TriFuse on both datasets.

TABLE VII
QUALITATIVE COMPARISON OF ARCHITECTURAL CHARACTERISTICS

Property	Feature ML	Single DL	Ensemble	TriFuse
Lexical modeling	✓	Partial	✓	✓
Semantic context	×	✓	✓	✓
Structural sequencing	×	Partial	Partial	✓
Cross-view interaction	×	×	×	✓
Representation fusion	×	×	×	✓
Attention weighting	×	×	×	✓
End-to-end learning	×	✓	×	✓

D. Comparison with State-of-the-Art

Table VII compares design choices at a high level. Many published systems vote or stack separate models after training; TriFuse shares layers and learns fusion weights inside the network.

Headline numbers from other papers are not directly comparable because splits, labels, and preprocessing differ. Within our own protocol, TriFuse usually beats the best neural baseline by 0.03–0.16 points in test accuracy. We therefore ran paired Wilcoxon and paired t -tests on the five Davidson fold accuracies stored in the project logs. Mean fold accuracy was 95.37% for TriFuse and 95.31% for Tuned LSTM; both tests returned $p > 0.05$ (Wilcoxon $p = 0.625$; paired t , $p = 0.79$). The tests therefore do not show a significant gap at the 5% level, which matches the tiny score differences in Tables IV and V.

E. Limitations

Six limitations should be noted. (i) Score gains over CNN, BiLSTM, and Tuned LSTM are small, and paired tests on Dataset II were not significant at $\alpha = 0.05$. (ii) Recent Transformer-heavy systems [14], [15], [16] are discussed qualitatively rather than retrained on our Shahane/Davidson splits. (iii) Dataset I mixes several annotation schemes into one abusive class, which helps coverage but blurs label semantics. (iv) A partially fine-tuned DistilBERT front end supplies token vectors, so pre-training contributes to the final score. (v) Only English binary labels are studied; finer moderation classes, images, and user metadata are out of scope. (vi) TriFuse is heavier to train than single-branch baselines, and we do not yet provide a detailed error study for sarcasm or borderline cases.

VIII. ABLATION STUDY

Ablation models remove or simplify parts of TriFuse while keeping the same splits, Focal Loss settings, optimizers, batch size, and epoch cap as the main runs.

TABLE VIII
BRANCH-LEVEL ABLATION RESULTS

Variant	Dataset I (Shahane)				Dataset II (Davidson)			
	Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1
Lexical Only	93.99%	93.80%	93.99%	93.88%	95.84%	95.90%	95.84%	95.87%
Semantic Only	92.99%	93.23%	92.99%	93.10%	95.46%	95.66%	95.46%	95.53%
Lexical + Semantic	94.08%	93.80%	94.08%	93.90%	95.68%	96.02%	95.68%	95.78%
Lexical + Structural	93.93%	93.68%	93.93%	93.77%	95.73%	95.79%	95.73%	95.76%
Semantic + Structural	94.18%	93.81%	94.18%	93.87%	95.65%	95.93%	95.65%	95.74%
Late Fusion	94.25%	93.89%	94.25%	93.82%	95.73%	95.96%	95.73%	95.81%
TriFuse (Full)	94.32%	94.03%	94.32%	94.12%	95.87%	95.96%	95.87%	95.91%

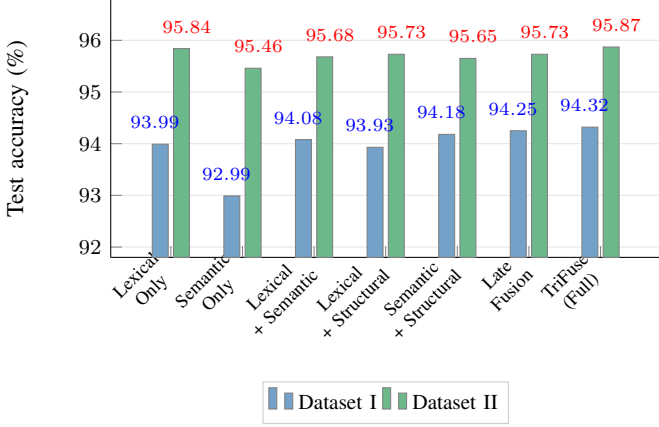


Fig. 8. Branch-level ablation results on Dataset I and Dataset II (test accuracy).

A. Branch-Level Contribution Analysis

Table VIII and Fig. 8 list ablation variants on both datasets. TriFuse (Full) records the highest accuracy in every column (94.32% on Dataset I and 95.87% on Dataset II), ahead of all single-branch, pairwise, and late-fusion variants.

B. Attention-Based Fusion Mechanism

Learned attention assigns different weights to the three branches before the softmax layer (Fig. 3). Against late fusion, TriFuse gains 0.14 points on Dataset II (95.87% vs. 95.73%) and 0.07 points on Dataset I (94.32% vs. 94.25%). Cross-view layers and the diversity loss explain part of that gap because late fusion trains the branches with less shared feedback. Training curves for the full model on Davidson appear in Fig. 5.

C. Summary of Findings

The ablation runs support four observations:

- 1) One branch alone misses part of the signal on each corpus.
- 2) Two-branch models score higher than single branches but stay below the full network.
- 3) Late fusion trails the learned fusion block on both datasets.
- 4) The complete TriFuse stack records the top accuracy in every column of Table VIII.

IX. CONCLUSION AND FUTURE WORK

We described TriFuse, a cyberbullying classifier that joins CNN, Transformer, and BiLSTM branches with cross-view modules and attention fusion. Focal Loss and auxiliary branch heads handle class skew during training.

On Shahane (Dataset I), test accuracy is 94.32% (F1 94.12%) and 5-fold CV is $94.29 \pm 0.13\%$, above all five baselines. On Davidson (Dataset II), test accuracy is 95.87% (F1 95.91%) and 5-fold CV is $95.37 \pm 0.23\%$ with the same

model order. Ablations confirm that removing branches or switching to late fusion lowers scores. The margins over strong neural nets are small, but the ranking holds on both corpora and across folds, which is the main argument for mid-level multi-view fusion in this setting, while acknowledging the limitations discussed in Section VII-E.

Next steps include multimodal inputs, attention plots for moderators, paired tests once Shahane fold logs are stored, stronger Transformer baselines on the same splits, and precision–recall analysis under deployment constraints.

STATEMENTS AND DECLARATIONS

Funding: This study received no external funding.

Conflict of Interest: The authors declare no competing interests.

Dataset Availability: Dataset I (Shahane) and Dataset II (Davidson) are public and cited as [10], [11]. No private data was collected or used for the creation of the dataset.

Authors' Contributions: Conceptualization: KM, SM, PG; formal analysis: KM, SM, PG; investigation: KM, SM, PG; methodology: KM, SM, PG; software: KM, SM; resources: KM, SM, PG; supervision: PG; validation: KM, SM, PG; writing, original draft: KM, SM; writing, review and editing: PG. All authors approved the final manuscript.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering and the Department of Electronics and Communication Engineering, Jaypee University of Information Technology, Waknaghat, for laboratory support during this study. Computing time on the IIIT Noida HPC cluster is also acknowledged.

REFERENCES

- [1] R. M. Kowalski, G. W. Giumetti, A. N. Schroeder, and M. R. Lattanner, "Bullying in the digital age: A critical review and meta-analysis of cyberbullying research among youth," *Psychological Bulletin*, vol. 140, no. 4, pp. 1073–1137, 2014. doi: 10.1037/a0035618.
- [2] Z. Waseem and D. Hovy, "Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter," in *Proc. NAACL-HLT*, San Diego, CA, USA, 2016, pp. 88–93. doi: 10.18653/v1/N16-2013.
- [3] P. Badjatiya, S. Gupta, M. Gupta, and V. Varma, "Deep learning for hate speech detection in tweets," in *Proc. WWW Companion*, Perth, Australia, 2017, pp. 759–760. doi: 10.1145/3041021.3054223.
- [4] M. Founta et al., "Large scale crowdsourcing and characterization of Twitter abusive behavior," in *Proc. ICWSM*, 2018, pp. 491–500. <https://doi.org/10.48550/arXiv.1802.00393>
- [5] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001. doi: 10.1023/A:1010933404324.
- [6] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems*, 2017.
- [7] Y. Kim, "Convolutional Neural Networks for Sentence Classification," in *Proc. of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014. doi: 10.3115/v1/D14-1181.
- [8] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- [9] A. Vaswani et al., "Attention Is All You Need," *arXiv preprint arXiv:1706.03762*, 2017. <https://doi.org/10.48550/arXiv.1706.03762>
- [10] S. Shahane, "Cyberbullying Dataset," Kaggle, 2020. [Online]. Available: <https://www.kaggle.com/datasets/saurabhshahane/cyberbullying-dataset>.

- [11] T. Davidson, D. Warmley, M. Macy, and I. Weber, "Automated Hate Speech Detection and the Problem of Offensive Language," in *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 11, no. 1, pp. 512–515, 2017. doi: 10.1609/icwsm.v11i1.14955.
- [12] M. A. Al-Garadi et al., "Predicting Cyberbullying on Social Media in the Big Data Era Using Machine Learning Algorithms: Review of Literature and Open Challenges," *IEEE Access*, vol. 7, pp. 70701–70718, 2019. doi: 10.1109/ACCESS.2019.2918354.
- [13] S. Agrawal and A. Awekar, "Deep learning for detecting cyberbullying across multiple social media platforms," in *Proc. ECIR*, 2018, pp. 141–153. doi: 10.1007/978-3-319-76941-7_11.
- [14] A. Muneer, A. Alwadain, M. G. Ragab, and A. Alqushaibi, "Cyberbullying Detection on Social Media Using Stacking Ensemble Learning and Enhanced BERT," *Information*, vol. 14, no. 8, p. 467, 2023. doi: 10.3390/info14080467.
- [15] F. Alrowais et al., "RoBERTaNET: Enhanced RoBERTa Transformer Based Model for Cyberbullying Detection with GloVe Features," *IEEE Access*, pp. 1–1, 2024. doi: 10.1109/ACCESS.2024.3386637.
- [16] M. Umer, E. Alabdulqader, A. Alarfaj, L. Cascone, and M. Nappi, "Cyberbullying Detection Using PCA Extracted GLOVE Features and RoBERTaNet Transformer Learning Model," *IEEE Transactions on Computational Social Systems*, pp. 1–10, 2024. doi: 10.1109/TCSS.2024.3422185.
- [17] J. Wang, K. Fu, and C.-T. Lu, "SOSNet: A Graph Convolutional Network Approach to Fine-Grained Cyberbullying Detection," in *2020 IEEE International Conference on Big Data (Big Data)*, Atlanta, GA, USA, 2020, pp. 1699–1708. doi: 10.1109/BigData50022.2020.9378065.
- [18] I. Rish, "An Empirical Study of the Naïve Bayes Classifier," in *IJCAI 2001 Workshop on Empirical Methods in Artificial Intelligence*, 2001.
- [19] Y. Li, M. Yang, and Z. Zhang, "A Survey of Multi-View Representation Learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 10, pp. 1863–1883, 2019. doi: 10.1109/TKDE.2018.2872063.
- [20] S. Ruder, "An Overview of Multi-Task Learning in Deep Neural Networks," *arXiv preprint arXiv:1706.05098*, 2017. <https://doi.org/10.48550/arXiv.1706.05098>
- [21] P. K. Verma, P. Agrawal, I. Amorim, and R. Prodan, "WELFake: Word Embedding Over Linguistic Features for Fake News Detection," *IEEE Transactions on Computational Social Systems*, vol. 8, no. 4, pp. 881–893, 2021. doi: 10.1109/TCSS.2021.3068519.
- [22] C. Suman, A. Naman, S. Saha, and P. Bhattacharyya, "A Multimodal Author Profiling System for Tweets," *IEEE Transactions on Computational Social Systems*, vol. 8, no. 6, pp. 1407–1416, 2021. doi: 10.1109/TCSS.2021.3082942.
- [23] T. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal Loss for Dense Object Detection," *IEEE Transactions on Pattern Analysis & Machine Intelligence*, vol. 42, no. 02, pp. 318–327, 2020. doi: 10.1109/TPAMI.2018.2858826.
- [24] D. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *International Conference on Learning Representations*, 2014. <https://doi.org/10.48550/arXiv.1412.6980>
- [25] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019. <https://doi.org/10.48550/arXiv.1711.05101>
- [26] S. Fati, A. Muneer, A. Alwadain, and A. Balogun, "Cyberbullying Detection on Twitter Using Deep Learning-Based Attention Mechanisms and Continuous Bag of Words Feature Extraction," *Mathematics*, vol. 11, no. 16, p. 3567, 2023. doi: 10.3390/math11163567.
- [27] J.-M. Xu, K.-S. Jun, X. Zhu, and A. Bellmore, "Learning from bullying traces in social media," in *Proc. of the 2012 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL HLT '12)*, USA, 2012, pp. 656–666. doi: 10.1145/2346676.2346686.
- [28] C. Wang, P. Nulty, and D. Lillis, "A Comparative Study on Word Embeddings in Deep Learning for Text Classification," in *Proc. of the 4th International Conference on Natural Language Processing and Information Retrieval (NLPIR '20)*, New York, NY, USA, 2021, pp. 37–46. doi: 10.1145/3443279.3443304.
- [29] N. Badri et al., "Combining FastText and GloVe word embedding for offensive and hate speech text detection," *Procedia Computer Science*, vol. 207, pp. 769–778, 2022. doi: 10.1016/j.procs.2022.09.132.
- [30] Z. Yang et al., "Hierarchical Attention Networks for Document Classification," in *Proc. of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, San Diego, CA, 2016, pp. 1480–1489. doi: 10.18653/v1/N16-1174.
- [31] M. I. Mahmud, M. Mamun, and A. Abdelgawad, "A Deep Analysis of Textual Features Based Cyberbullying Detection Using Machine Learning," in *2022 IEEE Global Conference on Artificial Intelligence and Internet of Things (GCAIoT)*, Alamein New City, Egypt, 2022, pp. 166–170. doi: 10.1109/GCAIoT57150.2022.10019058.